

FUNDAÇÃO EDSON QUEIROZ — UNIVERSIDADE DE FORTALEZA  
Disciplina: Programação Orientada a Objetos

# **Relatório Final do Projeto**

## **Sistema de RPG Orientado a Objetos**

*Modelagem de personagens, classes (arquétipos), guildas e inventário*

### **Alunos:**

Bruno de Abreu  
Luis Henrique

Data: 04/06/2026

## 1. Tema e Objetivo do Sistema

---

O presente projeto consiste no desenvolvimento de um sistema de RPG (Role-Playing Game) modelado integralmente em Java, sem o uso de bibliotecas externas que abstraíam os conceitos de orientação a objetos. O sistema representa o universo de um jogo de aventura, no qual personagens controláveis e não controláveis interagem em um mundo composto por guildas e itens.

O objetivo é demonstrar, de forma prática e coesa, a aplicação dos quatro pilares da Programação Orientada a Objetos (abstração, encapsulamento, herança e polimorfismo), além do uso de interfaces e de composição para a modelagem das dependências entre as entidades do jogo.

## 2. Introdução e Contextualização do Problema

---

Jogos de RPG são, por natureza, um domínio extremamente favorável à modelagem orientada a objetos: existem entidades com características em comum (todo personagem possui nome, nível, vida, mana, força e agilidade) e, ao mesmo tempo, especializações com comportamentos próprios (um Guerreiro luta corpo a corpo, enquanto um Mago ataca à distância com magias).

Representar essa hierarquia de forma procedural geraria grande duplicação de código e dificuldade de manutenção. A orientação a objetos permite descrever o que é comum em superclasses abstratas e o que é específico em subclasses concretas, além de padronizar comportamentos por meio de interfaces. O problema modelado, portanto, é o de organizar personagens, seus estilos de combate, suas posses (inventário/itens) e suas associações sociais (guildas) de maneira extensível e reutilizável.

## 3. Descrição Geral da Proposta

---

O sistema é estruturado a partir de uma entidade central e abstrata, *Personagem*, que define os atributos e comportamentos comuns a qualquer ser do jogo. A partir dela derivam duas linhas de especialização:

- Personagens jogáveis, representados pela classe abstrata *Arquetipos*, da qual herdam as classes concretas *Guerreiro*, *Arqueiro*, *Mago* e *Assassino*;
- Personagens não jogáveis (NPCs), representados pela classe *PersonagemNaoJogavel*, responsável por interações e missões.

Cada arquétipo jogável implementa uma interface que define seu estilo de combate: *CorpoACorpo* (*Guerreiro* e *Assassino*) ou *LongoAlcance* (*Arqueiro* e *Mago*). Além disso, todo personagem possui um *Inventario* (composição), que por sua vez agrega objetos do tipo *Item*. Personagens também podem ser agrupados em uma *Guilda*, que mantém uma coleção de membros.

## 4. Levantamento das Principais Entidades / Classes

---

O sistema é composto pelas seguintes classes e interfaces (13 tipos no total):

### Classes abstratas

- **Personagem** — superclasse abstrata de todos os personagens; concentra atributos (nome, nível, vida, mana, força, agilidade) e o inventário, além de comportamentos genéricos como *mover()*, *mostrarStatus()* e *usarHabilidadeEspecial()*.
- **Arquetipos** — subclasse abstrata de *Personagem* que representa os personagens jogáveis; adiciona *tipoClasse* e *armaPrincipal* e o método *uparNivel()*.

### Classes concretas (arquétipos jogáveis)

- **Guerreiro** — combatente corpo a corpo, com armadura e resistência física; habilidade `golpePesado()`.
- **Arqueiro** — combatente de longo alcance, com quantidade de flechas e precisão; habilidade `disparoMultiplo()`.
- **Mago** — combatente de longo alcance baseado em magia, com elemento mágico e poder mágico; habilidade `lançarMagia()`.
- **Assassino** — combatente corpo a corpo ágil, com chance de evasão e chance crítica; habilidade `ataqueDuplo()`.

### Outras classes concretas

- **PersonagemNaoJogavel** — NPC com função e diálogo; oferece `interagir()` e `mostrarMissao()`.
- **Guilda** — agrupa personagens; controla a quantidade de membros e oferece `adicionarMembro()` e `removerMembro()`.
- **Inventario** — armazena itens com capacidade máxima; oferece `adicionarItem()`, `removerItem()` e `listarItens()`.
- **Item** — objeto colecionável com nome, peso e valor; possui o método `usar()`.
- **Main** — classe principal, responsável por instanciar os objetos e demonstrar o funcionamento do sistema.

### Interfaces

- **CorpoACorpo** — define o contrato de combate corpo a corpo: `atacar(Personagem)` e `defender(int)`.
- **LongoAlcance** — define o contrato de combate à distância: `atacar(Personagem)` e `defender(int)`.

## 5. Funcionalidades Previstas

---

- Criar personagens jogáveis de quatro arquétipos distintos (Guerreiro, Arqueiro, Mago e Assassino), cada um com atributos e habilidade especial próprios;
- Criar personagens não jogáveis (NPCs) capazes de interagir e oferecer missões ao jogador;
- Executar ações de combate (atacar e defender) de acordo com o estilo de cada personagem;
- Subir de nível (`uparNivel`) e exibir o status atual do personagem (`mostrarStatus`);
- Gerenciar o inventário de cada personagem: adicionar, remover e listar itens, respeitando a capacidade máxima;
- Criar e administrar guildas, adicionando e removendo membros com controle de capacidade.

## 6. Regras de Negócio Principais

---

- Todo personagem é criado com um inventário próprio de capacidade igual a 10 itens;
- O inventário não permite adicionar itens além da capacidade máxima definida;
- A remoção de um item só ocorre se ele estiver de fato presente no inventário; caso contrário, o sistema avisa que o item não foi encontrado;
- Uma guilda possui capacidade máxima de membros e não aceita novos integrantes quando está cheia;
- A remoção de um membro só é efetuada se o personagem realmente pertencer à guilda;
- Cada arquétipo jogável é obrigado a definir seu próprio comportamento de ataque e defesa por meio da interface de combate que implementa.

## 7. Aplicação dos Pilares da Orientação a Objetos

---

- **Abstração** — as classes `Personagem` e `Arquetipos` são abstratas e representam entidades genéricas do jogo, não instanciáveis diretamente; as interfaces `CorpoACorpo` e `LongoAlcance` abstraem estilos de combate.
- **Encapsulamento** — todos os atributos das classes são declarados como `private` e o acesso a eles é feito exclusivamente por métodos `getters` e `setters` públicos.
- **Herança** — `Arquetipos` herda de `Personagem`; `Guerreiro`, `Arqueiro`, `Mago` e `Assassino` herdam de `Arquetipos`; `PersonagemNaoJogavel` herda de `Personagem`, especializando comportamentos genéricos.
- **Polimorfismo** — o método `usarHabilidadeEspecial()` é sobrescrito (sobreposição) em cada subclasse, produzindo comportamentos diferentes a partir da mesma assinatura; os métodos `atacar()` e `defender()` também variam conforme o arquétipo que implementa a interface.

### Recursos complementares

- **Interfaces** — duas interfaces distintas (`CorpoACorpo` e `LongoAlcance`) são implementadas por classes concretas, padronizando os contratos de combate.
- **Composição** — `Personagem` possui um `Inventario`, que por sua vez é composto por objetos `Item`; a `Guilda` é composta por um conjunto de `Personagem` — relações de dependência modeladas por composição/agregação.

## 8. Diagrama de Classes (UML)

A figura a seguir apresenta o diagrama de classes UML do sistema, evidenciando as relações de herança, implementação de interfaces e composição entre as entidades descritas acima.

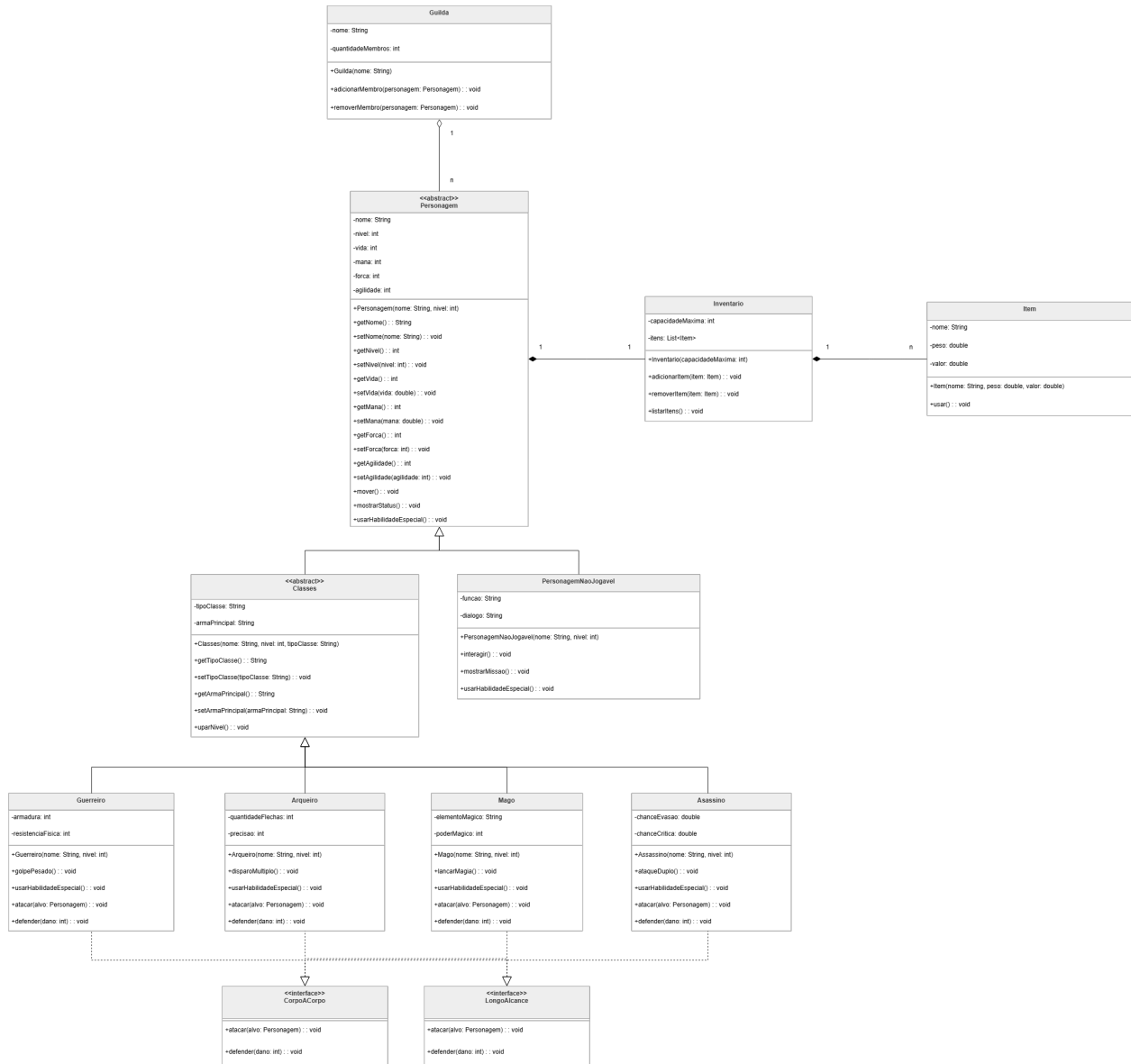


Figura 1 — Diagrama de Classes do Sistema de RPG.

## 9. Considerações Finais

---

O sistema de RPG desenvolvido atende aos requisitos mínimos do projeto: possui mais de dez classes, duas classes abstratas (Personagem e Arquetipos), duas interfaces distintas (CorpoACorpo e LongoAlcance) implementadas por classes concretas e aplica explicitamente os quatro pilares da orientação a objetos, além de empregar composição na modelagem das dependências entre as entidades.

A estrutura adotada favorece a extensibilidade: novos arquétipos de personagens, novos tipos de itens ou novas regras de combate podem ser incorporados com baixo impacto sobre o código existente, reaproveitando os contratos definidos pelas superclasses abstratas e pelas interfaces. As instruções de compilação e execução encontram-se no arquivo README.txt que acompanha o código-fonte.